

Практикум студента

Общий репозиторий и обмен изменениями

Цель: передать коммит через общий репозиторий и экспериментально различить получение истории и изменение рабочих файлов.

Сначала слушайте объяснение по слайдам, затем выполняйте шаги самостоятельно. На работу предусмотрено 60 минут. Все команды ниже вводятся в Git Bash, по одной строке; символ \$ вводить не нужно. Пример вывода не является командой.

Подготовка Windows 10 и 11

Скачайте 03_Стартовый_проект.zip. В Проводнике выберите «Извлечь всё». Откройте папку git-remote-p1. Работайте в отдельной новой папке внутри своих Документов; не распаковывайте поверх старой попытки. Запустите 00_OPEN_GIT_BASH.cmd.

Резервный запуск: правая кнопка по свободному месту папки → Open Git Bash here; в Windows 11 может понадобиться «Показать дополнительные параметры». Команды не вводятся в PowerShell, командной строке Windows или Блокноте. Файлы .sh запускайте через bash из Git Bash, как написано в шагах.

Если Git ещё не установлен: <https://git-scm.com/install/windows>. Выберите установщик для архитектуры компьютера, завершите установку и заново откройте Git Bash. Тип системы указан в Параметры → Система → О системе. Если установка недоступна, сообщите преподавателю: во время пары заполните объяснения и прогнозы, техническую часть завершите позднее. Это промежуточная, а не полная сдача.

Для вставки команды используйте Shift+Insert или меню правой кнопки. Кавычки должны быть прямыми. После случайного незакрытого выражения нажмите Ctrl+C и повторите одну строку. Перед каждым блоком сверяйте начальную папку; cd изменяет её для всех следующих команд.

Учебный сервис не запускается. Файлы config и README моделируют его конфигурацию. Общий репозиторий находится на том же диске: это модель обмена, не опыт сетевой отказоустойчивости.

A1 Проверка папки

Начальная папка: `git-remote-p1`. Все строки выполняйте по порядку в одном окне *Git Bash*.

```
pwd  
git --version  
ls
```

Ожидаемый результат

Путь заканчивается на `git-remote-p1`; видны `seed`, `evidence` и `pack.sh`.

Если результат отличается

Если вы внутри ZIP, сначала выполните «Извлечь всё». Если папка другая, откройте *Git Bash* из нужной папки.

A2 Общее хранилище и первая копия

Начальная папка: git-remote-p1. Все строки выполняйте по порядку в одном окне Git Bash.

```
git init --bare -b main team.git
git clone team.git alice
cd alice
git config user.name "Student Alice"
git config user.email "alice@example.invalid"
git config core.autocrlf false
git config core.quotePath false
cp ../seed/.gitattributes ../seed/README.txt ../seed/config.txt .
git add .gitattributes README.txt config.txt
git commit -m "Add baseline configuration"
git push -u origin main
git remote -v
```

Ожидаемый результат

В alice ветка main; первый коммит отправлен в team.git. Предупреждение empty repository при первом clone ожидаемо.

Если результат отличается

В user.name замените Student своим именем, сохраните Alice для различения ролей. Адрес учебный; регистрация не нужна. Если commit не создан, исправьте имя и повторите только commit и push.

В командах user.name замените Student своим именем, сохраняя Alice или Bob как роль. Адрес example.invalid — учебный пример. Не указывайте пароль, токен или данные чужой учётной записи.

А3 Вторая рабочая копия

Начальная папка: alice. Все строки выполняйте по порядку в одном окне Git Bash.

```
cd ..
git clone team.git bob
cd bob
git config user.name "Student Bob"
git config user.email "bob@example.invalid"
git config core.autocrlf false
printf "Service configuration\nMonitoring interval: 60 seconds.\n" > README.txt
git diff -- README.txt
git add README.txt
git commit -m "Document monitoring interval"
git push origin main
```

Ожидаемый результат

В bob сохранено и отправлено описание интервала. У общего main теперь два коммита.

Если результат отличается

Знак > намеренно заменяет небольшой учебный README. Если bob уже существует, не клонируйте повторно: сначала проверьте текущий шаг и историю.

A4 Получение без изменения файлов

Начальная папка: bob. Все строки выполняйте по порядку в одном окне Git Bash.

```
cd ../alice
cat README.txt
git rev-parse HEAD > ../evidence/head-before.txt
git fetch origin
git rev-parse HEAD > ../evidence/head-after-fetch.txt
git rev-parse origin/main > ../evidence/origin-after-fetch.txt
cat README.txt
git --no-pager log --oneline main..origin/main
git status -sb
```

Ожидаемый результат

README в alice ещё старый. HEAD до и после fetch одинаков; origin/main новее. log показывает Document monitoring interval; статус behind 1.

Если результат отличается

Если README уже обновлён, вероятно, вы в bob или ранее выполнили merge. Проверьте pwd. Не подменяйте сохранённые свидетельства вручную; опишите фактическое состояние.

Сравните три файла evidence. head-before.txt и head-after-fetch.txt должны совпасть, origin-after-fetch.txt должен отличаться. Не правьте эти файлы вручную. Зафиксируйте наблюдение до A5.

A5 Применение полученного изменения

Начальная папка: *alice*. Все строки выполняйте по порядку в одном окне *Git Bash*.

```
git merge --ff-only origin/main
cat README.txt
git status -sb
git rev-parse HEAD
git -C ../bob rev-parse HEAD
git --git-dir=../team.git rev-parse main
```

Ожидаемый результат

Fast-forward; в README появилась строка Monitoring interval: 60 seconds. Три хеша совпадают. Локальных изменений нет.

Если результат отличается

Если fast-forward невозможен, в *alice* появились собственные коммиты. Остановитесь, покажите log преподавателю. Не используйте force или reset для подгонки результата.

A6 Сохранение результата

Начальная папка: alice. Все строки выполняйте по порядку в одном окне Git Bash.

```
cd ..  
bash pack.sh
```

Ожидаемый результат

В submission подготовлены work.bundle, history.txt, state.txt, evidence и report.txt. Готовность истории не заменяет заполнение отчёта.

Если результат отличается

Если помощник сообщает ERROR, прочитайте условие и проверьте незавершённый шаг. Повторный pack обновляет технические файлы, но сохраняет существующий report.txt.

Отчёт и последующая сдача

1. В Проводнике откройте submission/report.txt через «Открыть с помощью» → Блокнот. Заполните все поля своими наблюдениями и нажмите Ctrl+S. Если этап не выполнен, укажите это прямо. Нельзя подставлять вымышленные хеши или сообщения.
2. Включите расширения файлов: Windows 10 → вкладка «Вид»; Windows 11 → «Вид» → «Показать» → «Расширения имён файлов». Проверьте, что отчёт называется report.txt, а не report.txt.txt.
3. Выделите содержимое submission: work.bundle, history.txt, state.txt, report.txt и папка evidence. Windows 11: «Сжать в ZIP-файл»; Windows 10: «Отправить» → «Сжатая ZIP-папка». Назовите внешний архив Фамилия_Обмен_Git.zip.
4. Откройте созданный ZIP и проверьте вложения. Сохраните архив и рабочие папки до проверки. Позднее загрузите один ZIP в задание ЭОС для соответствующей пары, когда преподаватель объявит ссылку и срок. Во время пары загрузка не требуется.

Оценивание из 10 баллов

Общее хранилище и две рабочие копии	2
Коммит второй роли отправлен обычным push	2
Сохранены наблюдения fetch и дано верное объяснение	2
Итоговые коммиты совпадают и деревья чистые	2
Bundle, свидетельства и заполненный отчёт представлены	2

По каждому критерию: 2 — полностью и верно; 1 — частично; 0 — отсутствует или неверно. Дополнительные осмысленные исправления объясните; число коммитов не является самостоятельным критерием.

Помощь и дополнительное наблюдение

Неверная папка

Выполните `pwd`. До перехода в `alice` или `bob` нахождение в корневой папке стенда нормально. Внутри `team.git` рабочие файлы не редактируют.

Автор не настроен

В текущей рабочей копии задайте `user.name` и `user.email`. Затем повторите только неудавшийся `commit`.

Папка уже существует

Вероятно, часть работы выполнена. Посмотрите состояние. Для новой попытки используйте другую папку; `.git` не удаляйте.

Рабочее дерево изменено

Посмотрите `git status` и `git diff`. Перед слиянием или упаковкой выясните, что изменилось. Не выполняйте `reset --hard` или очистку наугад.

Команда зависла в просмотре

Если `log` открыл просмотрщик, нажмите `q`. В практикуме используется `--no-pager`, чтобы вывод оставался в терминале.

Проверка не пройдена

Сообщите номер шага, `pwd`, точную команду и весь ответ Git. Сохраните папку. Не продолжайте выпуск при незавершённом слиянии.

Для завершивших раньше

В корне стенда выполните `git -C alice fetch origin` повторно. Сравните `HEAD` и `README` до и после. Объясните, почему новых изменений нет. Основную историю при этом не меняйте.

Официальная справка: <https://git-scm.com/docs/git-fetch> и <https://git-scm.com/docs/git-clone>